



DISEÑO DE INTERFACES WEB

UNIDAD DE TRABAJO 2

LENGUAJE DE HOJAS DE ESTILO CSS

1. [Introducción a CSS](#)
2. [Ejemplos de CSS](#)
3. [Medios](#)
4. [Modelo de cajas CSS](#)
5. [Herencia](#)

- 
- 
- 
- 
6. Unidades CSS
 7. Selectores. Clases e identificadores
 8. Tipos de elementos CSS. De bloque y en línea
 9. Box-Sizing
 10. Validaciones de código bien formado

1. INTRODUCCIÓN A CSS

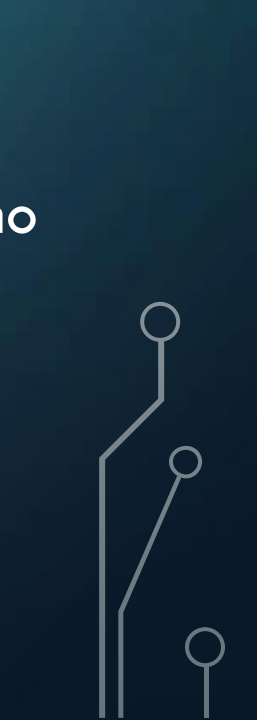
Cuando se trabaja con páginas web escritas en HTML y XHTML, es deseable que presenten un **estilo concreto** y un **aspecto definido**.

La forma más eficiente de realizar esto es distinguir entre la parte donde están los **contenidos**, los ficheros programados en HTML, y la parte del proyecto donde se recogen los aspectos relativos a la **apariencia**, archivos CSS (Cascading Style Sheets).

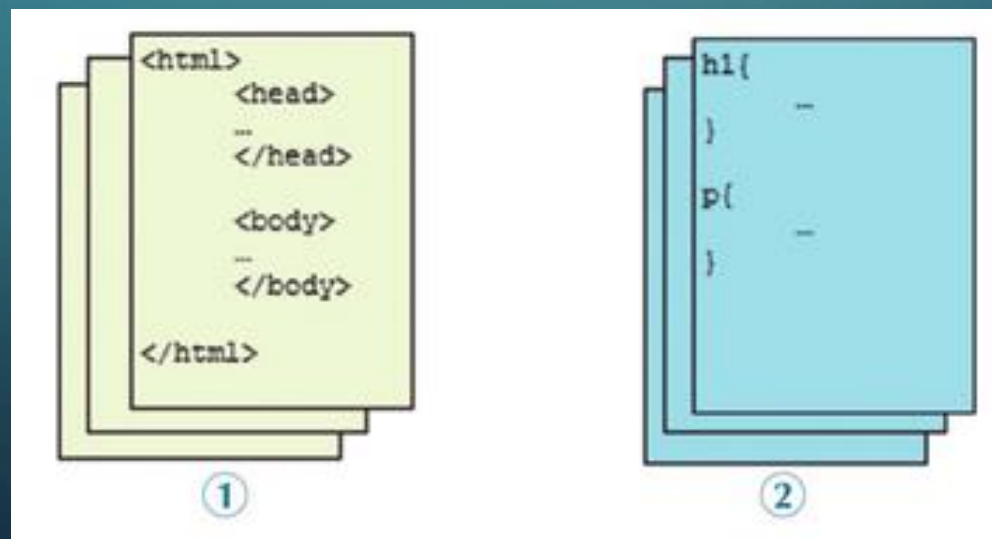


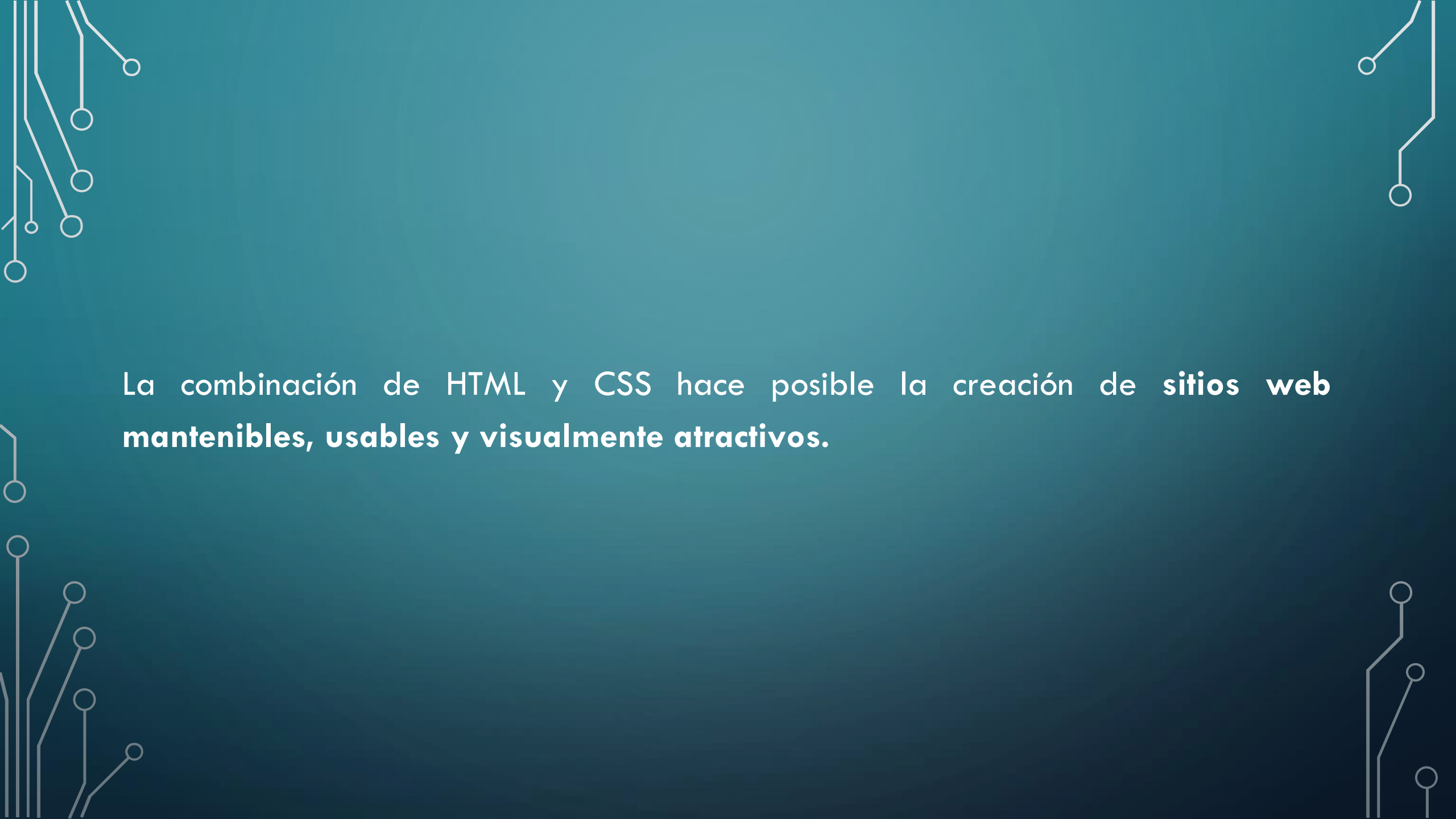
CSS es un lenguaje de hojas de estilo que será el encargado de controlar el aspecto de los sitios web.

Ventajas del uso de hojas de estilo CSS

- Mejora la accesibilidad del documento.
 - Reduce la complejidad de su mantenimiento, tanto si va a ser realizado por uno mismo como por terceros.
 - Facilita la visualización del sitio en dispositivos diferentes.
- 
- 

1. Creación de documentos estructurados en HTML/XHTML, donde se sitúan los contenidos, es decir, donde se crean los elementos de cada página (párrafo, titular, texto destacado, tabla, lista de elementos, ...)
2. Creación de hojas de estilo en lenguaje CSS, para definir el aspecto de cada elemento: color, tamaño y tipo de letra del texto, separación horizontal y vertical entre elementos, posición de cada elemento dentro de la página, ...



The background is a dark teal gradient. In the corners, there are white line-art illustrations of circuit boards or neural networks, with lines and small circles representing nodes and connections.

La combinación de HTML y CSS hace posible la creación de **sitios web mantenibles, usables y visualmente atractivos.**

ACTIVIDAD 1

2. EJEMPLOS DE CSS

Para añadir estilos a una página web basta con añadirlos en el código de la propia página web, en la cabecera, utilizando las etiquetas `<style>`, pero es recomendable hacerlo en documentos aparte.

Sintaxis de CSS

Para un elemento de HTML que se identifica a través de un selector, se indican una o varias propiedades de aspecto y los valores que se desean establecer.

Ej. para establecer la propiedad color del elemento body:

Regla

Selector{declaración; }

Declaración

Propiedad : Valor ;

Ejemplo

body {color: #333300;}

Ejemplo de CSS embebido

```
<!DOCTYPE html>

<html lang="en" xmlns="http://www.w3.org/1999/xhtml">
<head>
  <meta charset="utf-8" />
  <title>Ejemplo de estilos CSS en el propio documento</title>
  <style type="text/css">
    p {
      color: ■black;
      font-family: Verdana;
    }
  </style>
</head>
<body>
  <p> Este es un párrafo de prueba </p>
</body>
</html>
```



El principal inconveniente de tener código CSS mezclado con HTML es que, si se quiere hacer una modificación en los estilos definidos, es necesario ir página a página modificando el contenido de las tag `<style>`.



Lo recomendable es utilizar hojas de estilo externas que se cargan usando la etiqueta `<link>` de HTML.

Ejemplo de CSS externo

```
<!DOCTYPE html>

<html lang="en" xmlns="http://www.w3.org/1999/xhtml">
<head>
  <meta charset="utf-8" />
  <title>Ejemplo de estilos CSS en documento externo</title>
  <link rel="stylesheet" type="text/css" media="screen, handheld" href="./hojaEst.css"
</head>
<body>
  <p> Este es un párrafo de prueba </p>
</body>
</html>
```

Código CSS (hojaEst.css):

```
p {
  color: ■ black;
  font-family: verdana;
}
```

```
<link rel="stylesheet" type="text/css" media="screen, handheld" href="./hojaEst.css">
```

La función de cada uno de los atributos usados para enlazar el documento html con la hoja de estilos en CSS es:

- **rel:** tipo de relación que existe entre el recurso enlazado, el archivo CSS y la página HTML.

Para los archivos CSS, el valor es **stylesheet**.

```
<link rel="stylesheet" type="text/css" media="screen, handheld" href="._/hojaEst.css"
```

- **type:** tipo de recurso enlazado, en el caso de los archivos CSS su valor es **text/css**.
- **media:** tipo de medio en el que se van a aplicar los estilos del archivo CSS.
- **href:** URL del archivo CSS, que puede ser relativa o absoluta y puede apuntar a un recurso interno o externo al sitio web.

ACTIVIDAD 2

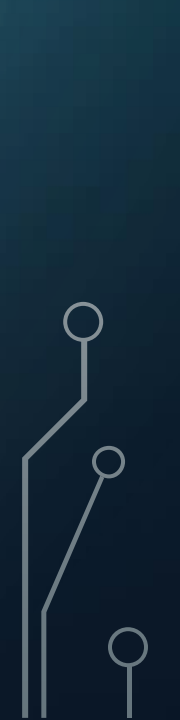
3. MEDIOS

Las hojas de estilo permiten especificar cómo se representa un documento, en función del tipo de medio de visualización.

Se denominan **medios** a los canales a través de los cuales se muestra la información del sitio web: auditivos, visuales, impresos, ...



Lo habitual es que se utilice una propiedad definida para cada tipo de canal.



En la actualidad, el medio más común es el utilizado para definir el aspecto del sitio web en pantalla, **screen**, y el que define la página para ser impresa, **print**.

Medio	Descripción
<i>all</i>	Todos los medios definidos.
<i>print</i>	Impresoras.
<i>braille</i>	Dispositivos táctiles que emplean el sistema braille.
<i>embosed</i>	Impresoras braille.
<i>handheld</i>	Dispositivos de mano: móviles, PDA, etc.
<i>projection</i>	Proyectores y dispositivos para presentaciones.
<i>screen</i>	Pantallas de ordenador.
<i>speech</i>	Sintetizadores para navegadores de voz utilizados por personas discapacitadas.
<i>tv</i>	Televisores y dispositivos con resolución baja.

3.1 @MEDIA

Si no se quiere definir toda una hoja CSS para cada uno de los medios de visualización, se pueden definir reglas en un mismo documento con @media.

Ej.: para el medio impresora se puede definir que los elementos <h1> se impriman con el tamaño de fuente especificado.

```
@media print {  
    h1 {  
        font-size: 15pt  
    }  
}
```


En una misma línea se pueden definir las reglas para varios medios de visualización.

```
@media screen, print {  
    h1 {  
        line-height: 1.2  
    }  
}
```

3.2 @IMPORT

```
@import "ficheroEstilo1.css" print;  
@import "ficheroEstilo2.css" screen;  
@import "ficheroEstilo3.css" print, screen;  
@import "ficheroEstilo4.css"; (Por defecto se aplica a todo)
```

Permite importar nuevas reglas desde otros ficheros CSS, e indicar el medio/s sobre los que debe aplicarse el nuevo estilo.

En primer lugar, aparece la URL que apunta al fichero CSS que contiene la guía de estilo, y luego se indica el medio sobre el que se va a aplicar.

```
@import "ficheroEstilo1.css" print;  
@import "ficheroEstilo2.css" screen;  
@import "ficheroEstilo3.css" print, screen;  
@import "ficheroEstilo4.css"; (Por defecto se aplica a todo)
```

Es importante tener en cuenta que estas reglas deben preceder a todas las demás reglas.

Si los datos de estilo del fichero se van a aplicar a varios medios al mismo tiempo, se indicará el nombre de estos separados por comas.

ACTIVIDAD 3

4. **MODELO DE CAJAS CSS**

El modelo de cajas es una de las características más importantes de las hojas de estilo.

Al crear una página en HTML, el uso de cada etiquetas simula lo que podría considerarse una caja, dotando a cada una de ellas de un identificador, indicado por el nombre de la etiqueta.

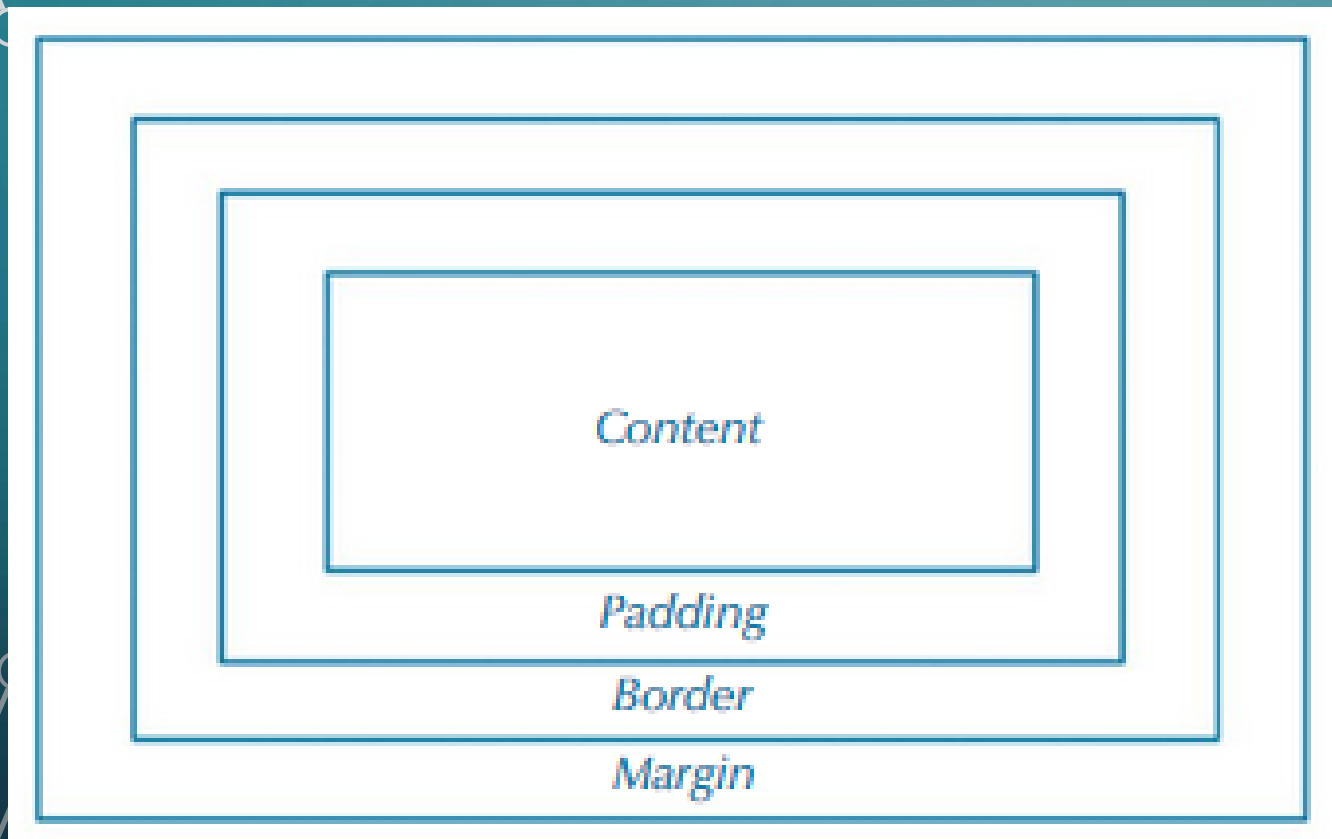


Este sistema de cajas permite, desde el fichero CSS, seleccionar cada caja y dotarle del estilo deseado.

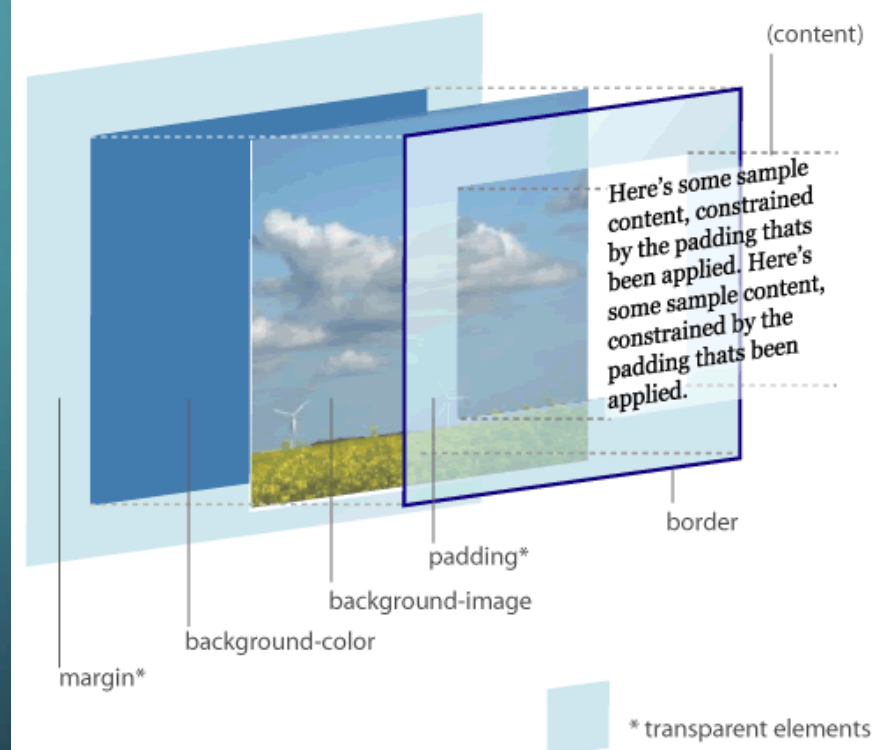


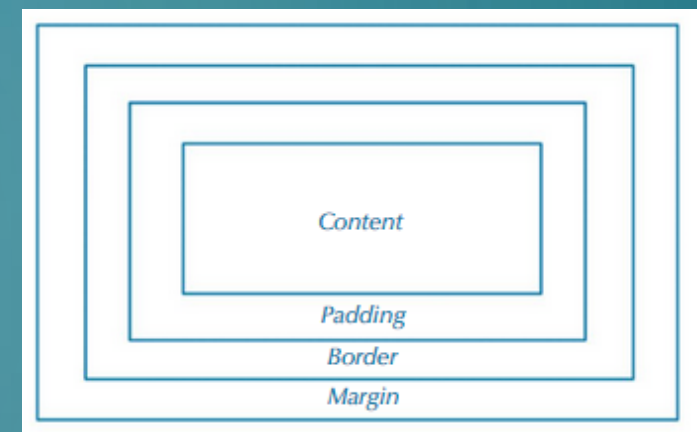
El uso de las guías de estilo con CSS permite modificar todas las características de cada una de las cajas creadas.

Análisis de cada una de estas partes



THE CSS BOX MODEL HIERARCHY





Las cajas de elementos, están hechas de cuatro componentes principales:

- **Content** (contenido del elemento): es lo que está en el núcleo de la caja.
- **Relleno** (padding): es el espacio que rodea al contenido.
- **Borde** (border): es la parte que perfila el relleno.
- **Margen** (margin): es el espacio que rodea al borde, la parte más externa del elemento.

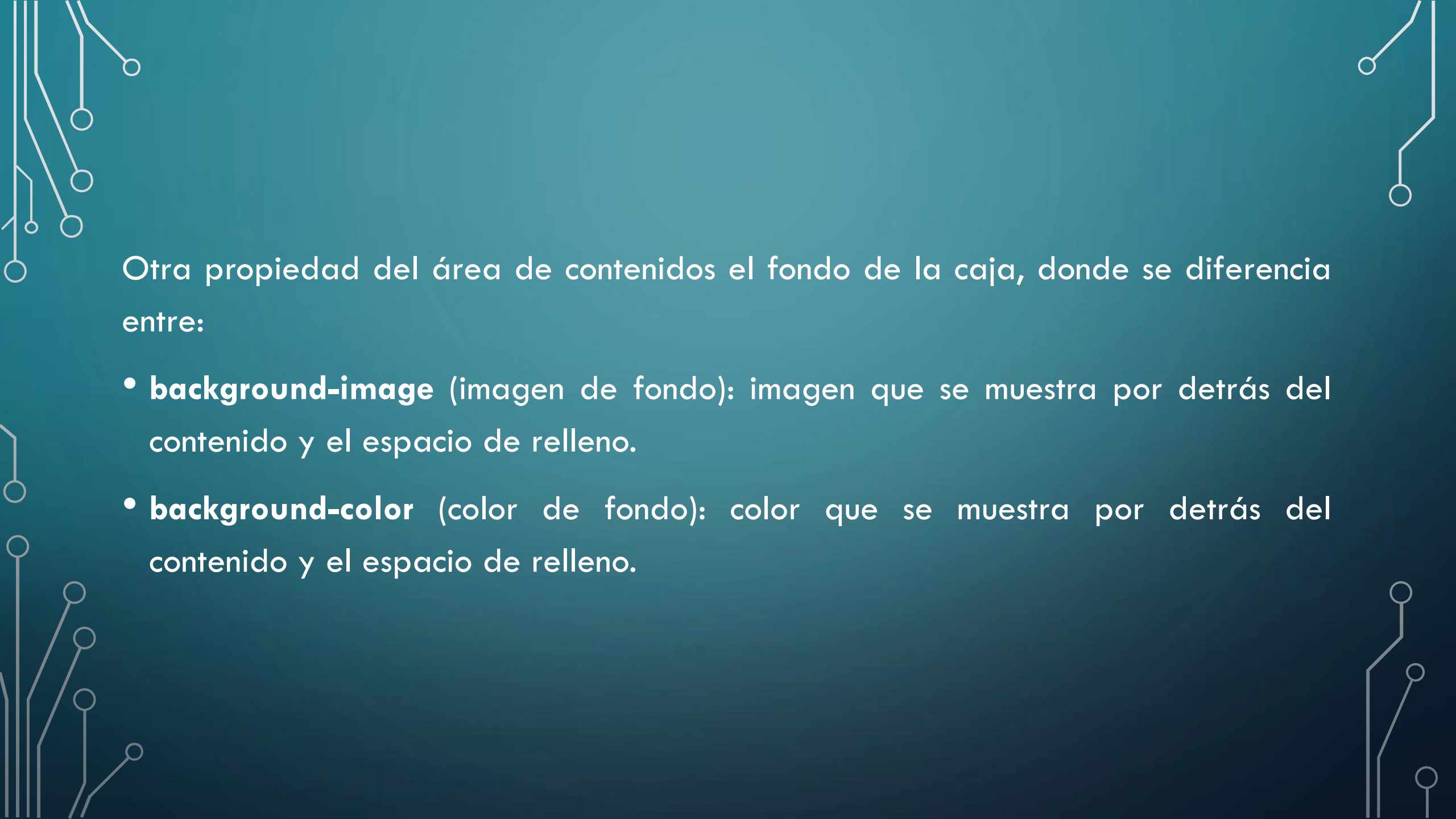
Las cajas de las páginas no son visibles a simple vista porque inicialmente no muestran ningún color de fondo ni ningún borde.

Ej. que muestra las cajas que forman una página web después de forzar a que todas las cajas muestren su borde:



4.1 WIDTH Y HEIGHT

Las propiedades principales con las que se trabaja en el modelo de cajas son **width** y **height**, relativas a las dimensiones de la caja de contenido o **content** y son el ancho y el alto del área donde se muestra contenido de la caja.



Otra propiedad del área de contenidos el fondo de la caja, donde se diferencia entre:

- **background-image** (imagen de fondo): imagen que se muestra por detrás del contenido y el espacio de relleno.
- **background-color** (color de fondo): color que se muestra por detrás del contenido y el espacio de relleno.

Ej. para cambiar el color del fondo de la caja y ancho y alto de un elemento **article**:

```
article {  
    background: #5DEAAB;  
    height: 100px;  
    width: 200px;  
}
```

Existen otras propiedades que permiten un manejo refinado del tamaño de la caja de contenido para limitar los tamaños.

Para ello, se usan las propiedades **min-width**, **max-width**, **min-height** y **max-height**.

4.2 PADDING

Es el área de relleno, es decir, el espacio que existe entre la caja de contenido y el borde.

Las características de esta caja que pueden modificarse son su altura y anchura.

Se distinguen cuatro zonas de acción: zona inferior, zona superior, zona izquierda y zona derecha.

El nombre de las propiedades que definen estas zonas son **padding-top**, **padding-right**, **padding-left** y **padding-bottom**.

```
article {  
  padding-top: 10px;  
  padding-right: 30px;  
  padding-bottom: 10px;  
  padding-left: 20px;  
}
```

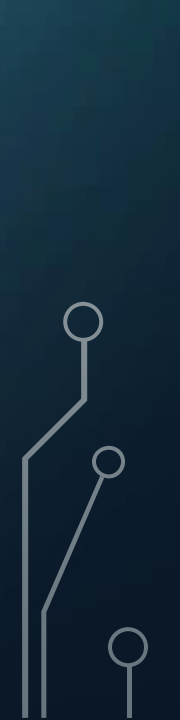
Si solo se pone **padding**, se aplicará el mismo formato a todos los extremos.

Ej. donde los límites inferior, superior, izquierdo y derecho tendrían un valor de 10 píxeles.

```
article {  
    padding: 10px;  
}
```



También es posible indicar los 4 tipos de **padding** (**top right bottom left**) en la misma fila, separando los valores por un espacio.



Cualquier color o imagen que se aplique de fondo al elemento se extenderá por el relleno.

4.3 BORDER

Esta zona es la que encierra el contenido y el relleno, **content** y **padding**.

Es posible modificar su grosor, estilo y color.

Es posible dar formato los cuatro extremos de la caja a la vez utilizando solo la palabra **border** o hacerlo de forma individual.

```
article {  
    border: 1px solid #000000;  
}
```

También se pueden establecer propiedades individuales de los distintos bordes de la caja de manera específica:

<i>border-top</i> <i>border-right</i> <i>border-bottom</i> <i>border-left</i>	Modifican a la vez el grosor, el estilo y el color de cada lado del borde.
<i>border-width</i> <i>border-style</i> <i>border-color</i>	Modifican de forma individual el grosor, el estilo y el color de los cuatros extremos a la vez.
<i>border-top-width</i>	Modifica el grosor del borde superior.
<i>border-top-style</i>	Modifica el estilo del borde superior.
<i>border-top-color</i>	Modifica el color del borde superior



Los bordes se generan con propiedades de estilo que especifican su estilo (ej. sólido), grosor y color.



Cuando el borde tiene huecos, el color o imagen de fondo aparecerá a través de esos huecos.



Diferentes tipos de **border-style**

TAREA 1

- Crear un página web donde se pueda ver un párrafo con cada tipo de estilo border.
- Aplica el grosor de borde deseado para cada párrafo.
- Añadir un color de fondo o una imagen de fondo en cada una de las cajas de los párrafos.

4.4 MARGIN

Es el margen que envuelve al resto de elementos CSS y que sostiene a otras cajas del diseño.

Las propiedades individuales son margin-top, margin-right, margin-bottom y margin-left.

```
article {  
    margin-top: 10px;  
    margin-right: 10px;  
    margin-bottom: 10px;  
    margin-left: 10px;  
}
```



Los márgenes siempre son transparentes $\Rightarrow$ el color del elemento padre se verá a través de ellos.



Al definir el largo de un elemento se está definiendo el largo del área de contenido y los largos de relleno, de borde y de márgenes se sumarían a esta cantidad.



Los márgenes tienen un comportamiento llamado **margin collapsing**, que cuando 2 cajas se tocan, la distancia entre ellas no es la suma de ambos márgenes, sino la del margen más grande.



El relleno, los bordes y los márgenes son opcionales, por lo que, si se ajustan a cero sus valores se eliminarán de la caja.



[Tutorial que ilustra el modelo de cajas y puede servir de apoyo](#)



TAREA 2

Esta es una caja de 250x200px con un borde rojo de 5px y un fondo gris claro, un relleno de 10px y un margen de 25px.

Esta es una caja de 300x200px con un borde verde de 7px, fondo gris, color de texto blanco, relleno de 5px y margen de 0px.

```
.caja1 {  
  border: 5px solid red;  
  background-color: lightgray;  
  padding: 10px;  
  margin: 25px;  
  width: 250px;  
  height: 200px;  
  ...  
}  
  
.caja2 {  
  border: 7px solid green;  
  background-color: gray;  
  padding: 5px;  
  margin: 0px;  
  width: 300px;  
  height: 200px;  
  color: white;  
  ...  
}
```

```
<div class="caja1">Esta es una caja de 250x200px con un borde rojo de 5px y un  
fondo gris claro, un relleno de 10px y un margen de 25px.</div>
```

```
<div class="caja2">Esta es una caja de 300x200px con un borde verde de 7px, fondo  
gris, color de texto blanco, relleno de 5px y margen de 0px.</div>
```


Esta es una caja con 50% de ancho y 10px desde la parte superior e izquierda dentro de un contenedor

Esta es una caja con 50% de ancho y 10px desde la parte superior y 50px a la izquierda dentro de un contenedor

Esta es una caja con 50% de ancho y 10px desde la parte superior y 100px a la izquierda dentro de un contenedor

...

```
<div class="contenedor">  
  <div class="caja1">Esta es una caja con 50% de ancho y 10px desde la parte  
superior e izquierda dentro de un contenedor</div>  
  <div class="caja2">Esta es una caja con 50% de ancho y 10px desde la parte  
superior y 50px a la izquierda dentro de un contenedor</div>  
  <div class="caja3">Esta es una caja con 50% de ancho y 10px desde la parte  
superior y 100px a la izquierda dentro de un contenedor</div>  
  ...  
</div>
```

```
.contenedor {  
  border: 5px solid red;  
}  
  
.caja1 {  
  border: 5px solid green;  
  margin-top: 10px;  
  margin-left: 10px;  
  width: 50%;  
}  
  
.caja2 {  
  border: 5px solid blue;  
  margin-top: 10px;  
  margin-left: 50px;  
  width: 50%;  
}  
  
.caja3 {  
  border: 5px solid orange;  
  margin-top: 10px;  
  margin-left: 100px;  
  width: 50%;  
}
```

Esta es una caja sin relleno

Esta es una caja con 25px de relleno

Esta es una caja con 50px de relleno

```
<div class="caja1">
  Esta es una caja sin relleno
</div>
<div class="caja2">
  Esta es una caja con 25px de relleno
</div>
<div class="caja3">
  Esta es una caja con 50px de relleno
</div>
```

```
.caja1 {
  border: 5px solid red;
  margin: 10px;
  ...
}

.caja2 {
  border: 5px solid blue;
  margin: 10px;
  padding: 25px;
  ...
}

.caja3 {
  border: 5px solid green;
  margin: 10px;
  padding: 50px;
  ...
}
```



Esta es una caja con diferentes tipos de bordes

Esta es otra caja con diferentes tipos de bordes

Esta es una caja con diferentes tipos de bordes dentro de otra caja

Estas son cajas dentro de cajas



```
<div class="caja1">Esta es una caja con diferentes tipos de bordes</div>
<div class="caja2">Esta es otra caja con diferentes tipos de bordes</div>
<div class="caja1">
  <div class="caja2">Esta es una caja con diferentes tipos de bordes dentro de otra
  caja</div>
</div>
<div class="caja3">
  <div class="caja3">
    <div class="caja3">
      <div class="caja3">
        <div class="caja3">
          Estas son cajas dentro de cajas
        </div>
      </div>
    </div>
  </div>
</div>
</div>
```

```
.caja1 {
  border-top: 5px dotted green;
  border-right: 5px dashed orange;
  border-bottom: 5px dotted red;
  border-left: 5px dashed blue;
  padding: 15px;
  margin-top: 15px;
  margin-bottom: 15px;
  text-align: center;
  ...
}

.caja2 {
  border: 5px solid black;
  border-left-width: 10px;
  border-right-width: 10px;
  border-top-color: aqua;
  border-bottom-color: pink;
  padding: 15px;
  text-align: center;
  ...
}

.caja3 {
  border: 2px solid black;
  padding: 10px;
  ...
}
```

ACTIVIDAD 4

5. HERENCIA

Es el mecanismo mediante el cual determinadas propiedades de un elemento **padre** se transmiten a sus **hijos**.

No todas las propiedades CSS son heredables; los márgenes no se heredan porque es poco probable que un elemento hijo necesite los mismos márgenes que su padre.

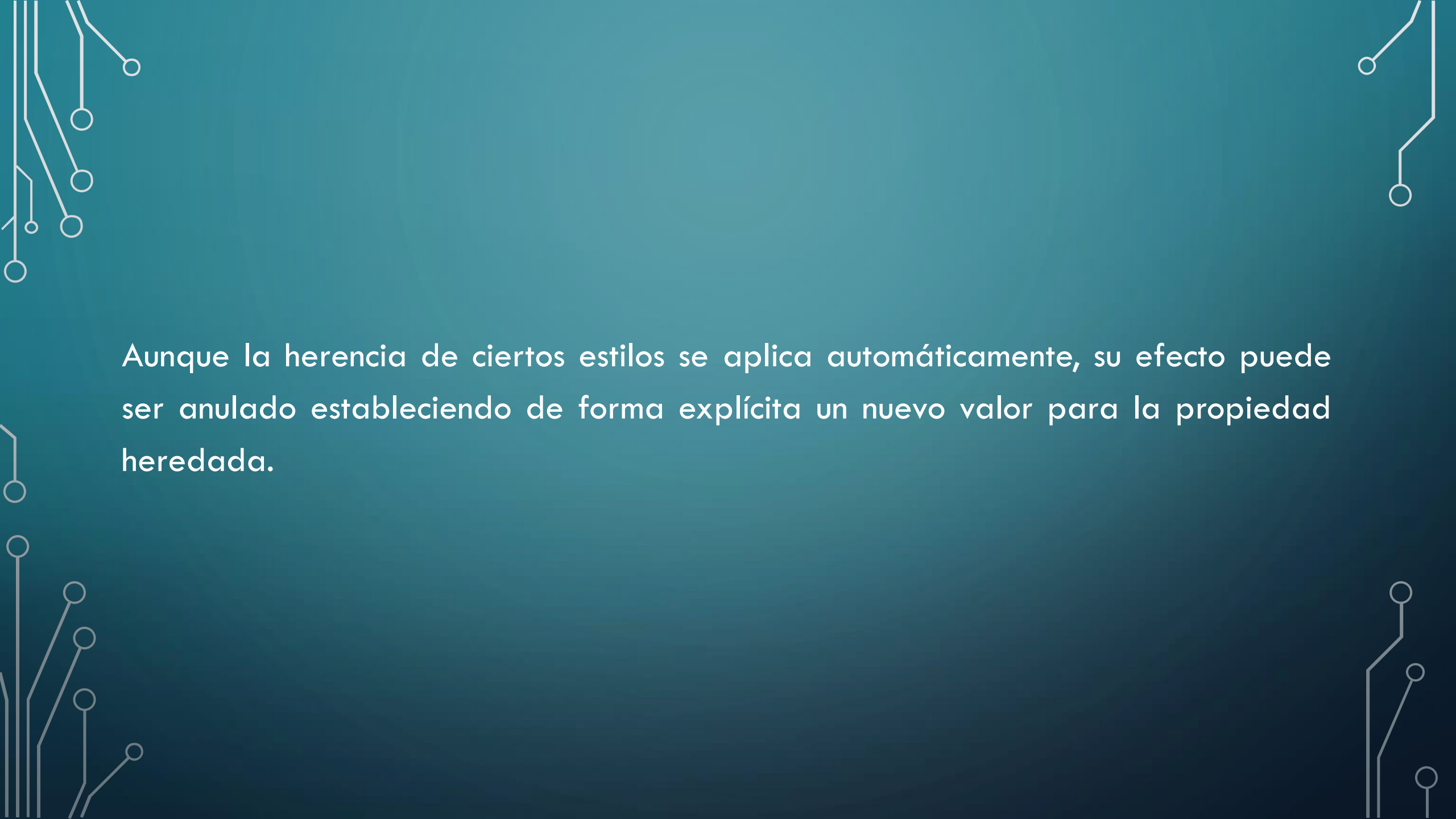
Cuando se establece el valor de una propiedad CSS en un elemento, sus elementos descendientes heredan de forma automática el valor de esa propiedad.

Ej.: si cambiamos el color del texto del selector body sus elementos descendientes o hijos que se incluyan en él como pueden ser un título h1 y un párrafo p, heredarán el color.

```
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style media="screen">
    body{
      color: red;
    }
  </style>
</head>
<body>
  <h1>Título con herencia de color</h1>
  <p>Esto es mi párrafo con herencia de color</p>
</body>
```

Título con herencia de color

Esto es mi párrafo con herencia de color

The background is a dark teal gradient. In the corners, there are white line-art illustrations of circuit boards or neural networks. These consist of vertical and horizontal lines of varying lengths, some ending in small circles, creating a technical or digital aesthetic.

Aunque la herencia de ciertos estilos se aplica automáticamente, su efecto puede ser anulado estableciendo de forma explícita un nuevo valor para la propiedad heredada.

Propiedades CSS heredables por defecto en los elementos HTML

Resumen de las propiedades heredadas por defecto en HTML

color	text-align
font	visibility
letter-spacing	white-space
direction	border-collapse
word-spacing	quotes
line-height	list-style-image



Se puede forzar que ciertas propiedades que en principio no se heredan, si sean heredadas del padre asignándolas uno de los siguientes valores:

- **inherit:** aplica el valor de la propiedad del elemento padre.
 - **initial:** aplica el valor que presenta inicialmente la propiedad.
 - **unset:** hereda el valor de la propiedad del elemento padre; si este no existe, aplica el valor inicial.
- 
- 

ACTIVIDAD 5

6. UNIDADES CSS

CSS tiene varias unidades diferentes para expresar tamaños.

Muchas de las propiedades CSS toman valores de tamaño o longitud, como width, margin, padding, font-size, ...

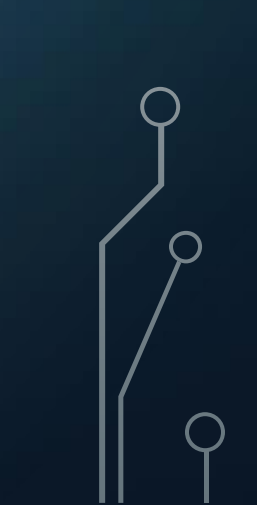


El tamaño o longitud se expresa como un número seguido de una unidad de longitud, 10px, 2em, ...

No puede aparecer un espacio en blanco entre el número y la unidad, sin embargo, si el valor es 0, la unidad se puede omitir.

Para algunas propiedades CSS, se permiten longitudes negativas.

Hay dos tipos de unidades de longitud: **absolutas y relativas**.



6.1 LONGITUDES ABSOLUTAS

Las unidades de longitud absoluta son fijas y una longitud expresada en cualquiera de estas aparecerá exactamente como ese tamaño.

No se recomienda el uso de unidades de longitud absoluta en la pantalla porque los tamaños de pantalla varían mucho.

Sin embargo, se pueden utilizar si se conoce el medio de salida, como para el diseño de impresión.

Unidades de este tipo son cm, mm, in(inches), px(pixels), pt(points), ...

Unit	Description
cm	centimeters
mm	millimeters
in	inches (1in = 96px = 2.54cm)
px *	pixels (1px = 1/96th of 1in)
pt	points (1pt = 1/72 of 1in)
pc	picas (1pc = 12 pt)

* Los píxeles (px) son relativos al dispositivo de visualización.

Para dispositivos de baja resolución, 1 px es un píxel (punto) del dispositivo de la pantalla y para impresoras y pantallas de alta resolución, 1 px implica varios píxeles del dispositivo.

6 .2 LONGITUDES RELATIVAS

Las unidades de longitud relativa especifican una longitud relativa a otra propiedad de longitud.

Las unidades de longitud relativa se escalan mejor entre diferentes medios de renderizado.

Unit	Description
em	Relative to the font-size of the element (2em means 2 times the size of the current font)
ex	Relative to the x-height of the current font (rarely used)
ch	Relative to the width of the "0" (zero)
rem	Relative to font-size of the root element
vw	Relative to 1% of the width of the viewport*
vh	Relative to 1% of the height of the viewport*
vmin	Relative to 1% of viewport's* smaller dimension
vmax	Relative to 1% of viewport's* larger dimension
%	Relative to the parent element

- Viewport = el tamaño de la ventana del navegador. Si la ventana tiene 50 cm de ancho, $1\text{ vw} = 0,5\text{ cm}$.

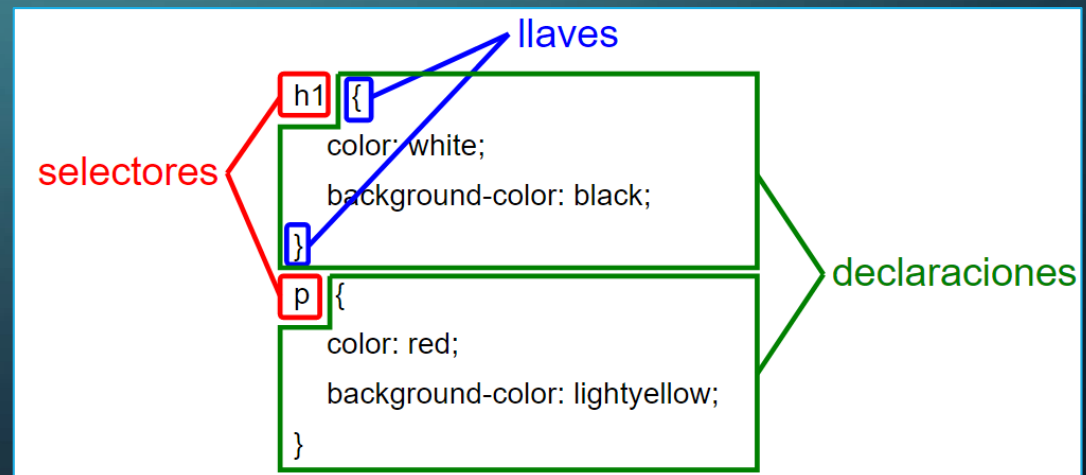
IMPORTANTE: las unidades em y rem son las más prácticas para crear diseños perfectamente escalables que se redimensionen correctamente al visualizarse en diferentes dispositivos con distintos tamaños de pantalla.

Unidades de medida en CSS para especificar tamaños

7. SELECTORES. CLASES E IDENTIFICADORES

Para poder hacer el diseño de la página existen los mecanismos denominados **reglas, selectores y declaraciones**.

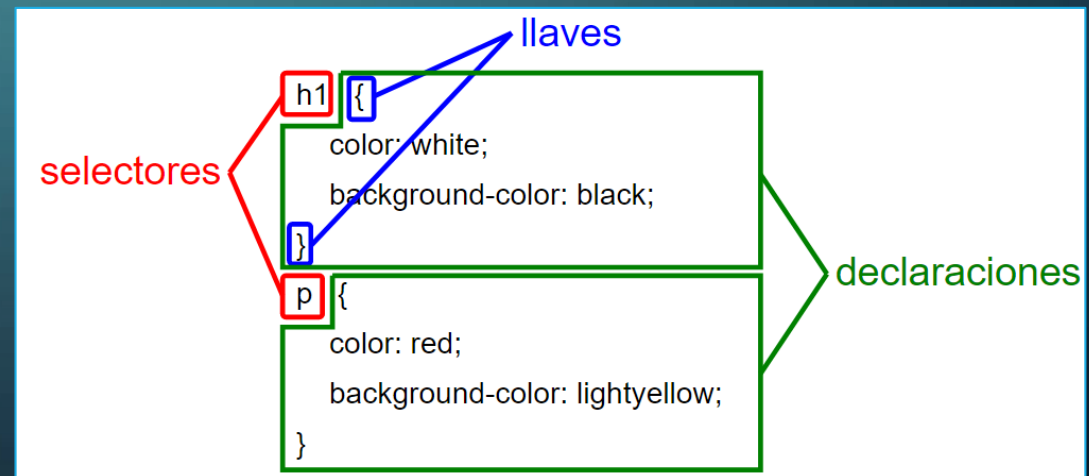
Cada regla o conjunto de reglas consiste en uno o más selectores y un bloque de declaración o bloque de estilo.



En el interior de estos bloques se definen elementos de aplicación, sobre los cuales se da valor a las propiedades deseadas.

Los estilos se aplican a los elementos del documento que cumplan con el Selector que les precede.

Cada bloque de estilo se define entre llaves, y está formado por una o varias declaraciones de estilo con la sintaxis.

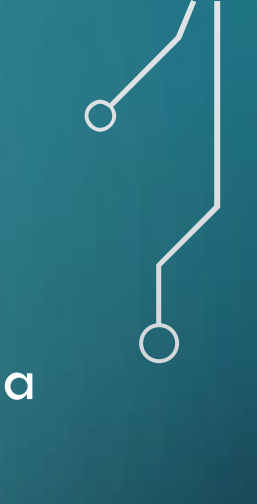


La declaración indica **lo que hay que hacer**, y el selector indica **sobre qué término hay que hacerlo**.

Los selectores son imprescindibles para aplicar de forma correcta los estilos CSS en una página.

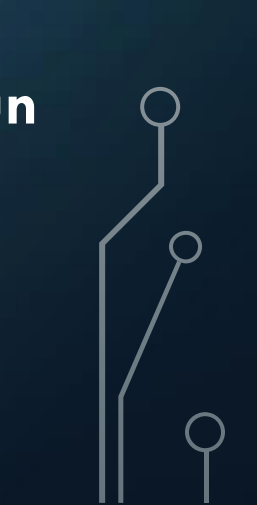
```
h1 {  
  color: white;  
  background-color: black;  
}
```

The diagram illustrates the components of a CSS declaration block. It shows a selector 'h1' followed by an opening curly brace '{'. Inside the block, there are two declarations: 'color: white;' and 'background-color: black;'. Each declaration consists of a property name, a colon, a value, and a semicolon. The closing curly brace '}' is at the end. Labels with arrows point to these components: 'propiedad' points to 'color' and 'background-color'; 'dos puntos' points to the colon in the first declaration; 'espacio en blanco' points to the space between the colon and the value 'white'; 'valor' points to 'white'; 'punto y coma' points to the semicolon in the first declaration.



A un mismo elemento HTML se le pueden asignar infinitas reglas CSS y cada regla CSS puede aplicarse a un número infinito de elementos.

Una misma declaración puede aplicarse sobre varios selectores y un mismo selector se puede utilizar en varias reglas.



Los selectores son el abc del CSS y hay que conocerlos a fondo para crear un código profesional.

7.1 SELECTORES BÁSICOS

Existen múltiples tipos de selectores, en función del rango de acción sobre el que modifican las propiedades de los elementos que se definen en su declaración, como:

- Selector universal
- Selector etiqueta
- Selector descendente
- ...

7.1.1 SELECTORES UNIVERSAL - *

Este selector permite seleccionar todos los elementos de una misma página, es decir, se da el mismo formato a todos los elementos recogidos en la página que esté utilizando esa hoja de estilo CSS.

```
*{  
    font-size: 3em;  
    color: red;  
    text-align: left;  
}
```

7.1.2 SELECTOR ETIQUETA

Permite seleccionar todos los elementos que contenga la etiqueta indicada en dicho selector.

Así, todos los elementos de un mismo tipo presentarán un mismo formato, manteniendo una apariencia uniforme en todo el diseño del sitio web.

```
etiquetaHTML{  
    propiedad: valor;  
}
```

Ej.: Se define un estilo diferenciado para los bloques determinados a través de las etiquetas de encabezado `<h1>` y `<h2>`.

Los elementos de tipo `h1` estarán alineados a la izquierda y serán de color rojo, mientras que a los de tipo `h2` tendrán una alineación central y serán de color azul.

```
h1{
    font-size: 3em;
    color: red;
    text-align: left;
}

h2{
    font-size: 2em;
    color: blue;
    text-align: center;
}
```

Es posible indicar los selectores de manera individual o se pueden encadenar varias etiquetas HTML sobre las que se quiere aplicar el mismo estilo y luego incorporar particularidades para cada selector.

Selectores separados		
<pre>p{ ... }</pre>	Selecciona todos los párrafos de la página	<pre>h1 { color: red; }</pre>
<pre>h1{ ... }</pre>	Selecciona todos los títulos de tipo h1 de la página	<pre>h2 { color: blue; }</pre>
<pre>h2{ ... }</pre>	Selecciona todos los títulos de tipo h2 de la página	<pre>p{ color: black; }</pre>

Selectores encadenados

Se aplica el mismo estilo a todos los elementos representados por las etiquetas encadenadas. A continuación, se indica el formato que se le va a aplicar de la misma forma que en los selectores separados.

```
h1,h2,h3{
color: #8A8E27;
font-weight:normal;
font-family: Arial, Helvetica, sans-serif;
}
```

Selectores encadenados incorporando particularidades para cada selector

Al igual que en el caso anterior se aplica el mismo estilo a todos los elementos representados por las etiquetas encadenadas, ya continuación, se indica las particularidades de cada selector.

```
h1,h2,h3{
color: #8A8E27;
font-weight:normal;
font-family: Arial, Helvetica, sans-serif;
}

h1{
font-size: 2em;
}

h2{
font-size: 1.5em;
}

h3{
font-size: 1.2em;
}
```

7.1.3 SELECTOR DESCENDENTE

Se utiliza cuando se quiere aplicar el estilo a los descendientes de un selector dado.

Sintaxis:

```
selector1 selector 2 ... selectorNhtml{  
    propiedad: valor;  
}
```

Ej.: el estilo se aplica a los dos enlaces, puesto que ambos se localizan bajo un selector de tipo `<p>`:

```
p a {  
  color:red;  
}
```

```
<p>  
  <a href="#">enlace uno</a>  
</p>  
<p>  
  <span>  
    <a href="#">enlace dos</a>  
  </span>  
</p>
```

https://www.w3schools.com/tags/tag_span.asp

Se pueden encadenar tantos selectores descendentes como niveles haya en el HTML.

Ej.: el estilo se aplicará al segundo enlace, dado que es hijo de `<p>` y de `<span>`:

```
p span a {  
  color:red;  
}
```

```
<p>  
  <a href="#">enlace uno</a>  
</p>  
<p>  
  <span>  
    <a href="#">enlace dos</a>  
  </span>  
</p>
```


Se pueden combinar selectores descendentes con el selector universal *.

Ej.: se mostrará en rojo sólo el segundo enlace, dado que el estilo se aplica sobre todos los elementos de tipo `<a>` que se encuentren dentro de cualquier elemento que, a su vez, se encuentre dentro de `<p>`.

```
p * a {  
  color:red;  
}
```

```
<p>  
  <a href="#">enlace uno</a>  
</p>  
<p>  
  <span>  
    <a href="#">enlace dos</a>  
  </span>  
</p>
```

ACTIVIDAD 6

7.2 SELECTORES DE CLASE E IDENTIFICADORES

A veces se quiere aplicar un mismo estilo a algunos y no a todos los elementos de un cierto tipo, para ello, se utilizan los atributos de HTML **id** y **class**, que permiten **agrupar o identificar al elemento concreto o grupo de elementos a los que se aplicará el estilo.**

7.2.1 SELECTOR USANDO IDENTIFICADOR - ID

Los selectores basados en identificadores seleccionan un atributo al que dar formato buscando en el contenido de los atributos **id** del documento estructurado en HTML.

La llamada al identificador desde la hoja de estilo se hace precedida del carácter **#**.

Los selectores id son únicos, sólo puede existir uno con el mismo nombre de atributo para toda la página web.

```
#identificador{          →  <etiqueta id = identificador>
    propiedad: valor;      ...
}                          </etiqueta>
```

Ej.: se aplica el estilo a todos los elementos HTML con el atributo id = "anexos".

```
#anexos {
    font-size: 18px; text-align: center;
}
```

```
<h1 id="anexos">
    Anexos 1
</h1>
<h2 id="anexos">
    Anexos 2
</h2>
<h2 id="textos">
    Textos
</h2>
```

Existe la posibilidad de utilizar el identificador de una forma más concreta, como **identificador dependiente**, es decir, indicando también el tipo de elemento HTML que tiene el atributo id definido.

Ej.: sólo el primer título <h2> se mostraría en color azul.

```
h2#anexos {  
    color:blue  
}
```



Se recomienda que el valor del id sea un nombre único que caracterice, de forma breve y esquemática al elemento y que, además, sea fácilmente reconocible por el programador.



Se utiliza con frecuencia para identificar las secciones principales de las páginas: contenido, cabecera, pie, ...

7.2.2 IDENTIFICADOR USANDO CLASE - CLASS

Se emplea el atributo **class** para identificar distintos elementos como parte de un grupo conceptual y así poder modificarlos con una única regla de estilo.

Ej.: como dos elementos distintos se clasifican de la misma forma mediante la asignación del valor "especial" al atributo **class**.

```
.especial{  
  font-style:  oblique;  
}  
h1.especial {  
  letter-spacing: 5px;  
}
```

```
<h1 class="especial">Importante! esto es un Titulo especial</h1>  
<p class="especial"> Hoy tenemos cosas especiales para ti</p>
```

Importante! esto es un título especial

Hoy tenemos cosas especiales para ti

ACTIVIDAD 7

7.2.3 RECORDANDO EL ELEMENTO <DIV>

La etiqueta o elemento `<div>` define una división o una sección en un documento HTML.

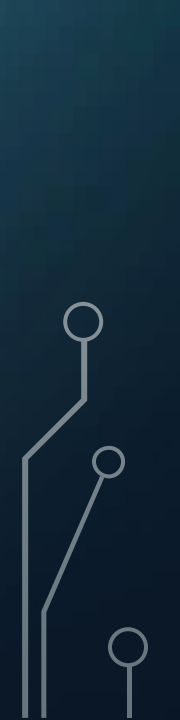
Se utiliza como **contenedor** de elementos HTML, que luego se estiliza con CSS o se manipula con JavaScript $\Rightarrow$ el elemento `<div>` puede ayudar a organizar el código HTML agrupando elementos que conceptualmente tienen algo en común.

Y para dar estilo a esos elementos de manera conjunta se pueden usar selectores `class` e `id`.



Nota: De forma predeterminada, los navegadores siempre colocan un salto de línea antes y después del elemento `<div>`, es lo que se denomina un **Elemento de bloque**, como `h1`, `p`, ...

https://www.w3schools.com/tags/tag_div.ASP



ACTIVIDAD 8

7.3 SELECTORES AVANZADOS

7.3.1 SELECTOR HIJO

Se usará este selector cuando se quiera aplicar el estilo a los **hijos** de un selector.

Sintaxis:

```
selector1 > selector2{  
    propiedad: valor;  
}
```

Las propiedades afectan a los elementos con la segunda etiqueta contenidos dentro de la primera etiqueta pero no les afecta si hay etiquetas intermedias entre ellos.

Ej.: únicamente los párrafos `<p>` dentro de una división `<div>` sin elementos intermedios se ven de color rojo.

```
div {  
  border: red 3px solid;  
  margin: 10px;  
  padding: 10px;  
}  
  
div > p {  
  color: red;  
}
```

Esto es un párrafo `<p>`.

Esto es un párrafo `<p>`.

- Esto es una lista

Esto es un párrafo `<p>`
dentro de una lista

Esto es otro párrafo `<p>`.

*Esto no es un párrafo `<p>`, sino
una dirección `<address>`.*

Esto es otro párrafo `<p>`.

7.3.2 SELECTOR CONSECUTIVO

Se usará este selector cuando se quiera aplicar el estilo a un elemento situado a continuación de otro.

Sintaxis:

```
selector1 + selector2{  
    propiedad: valor;  
}
```

Ej.: únicamente el párrafo `<p>` inmediatamente posterior a la división `<div>` se ve de color rojo.

```
div {  
  border: red 3px solid;  
  margin: 10px;  
  padding: 10px;  
}  
  
div + p {  
  color: red;  
}
```

Esto es un párrafo `<p>`.

Esto es un párrafo `<p>`.

- Esto es una lista

Esto es un párrafo `<p>` dentro de una lista

Esto es otro párrafo `<p>`.

Esto no es un párrafo `<p>`, sino una dirección `<address>`.

Esto es otro párrafo `<p>`.

Esto es otro párrafo `<p>`.

7.3.3 SELECTOR HERMANOS

Si se escriben dos etiquetas seguidas separadas por una virgullita, ~, las propiedades afectan únicamente a los elementos con la segunda etiqueta que van después de un elemento con la primera etiqueta, siendo ambos elementos hermanos, es decir, hijos del mismo padre.

```
selector1 ~ selector2{  
    propiedad: valor;  
}
```

Ej.: se ven de color rojo dos de los párrafos <p>.

Dentro de la división, el segundo párrafo se muestra de color rojo aunque haya una lista entre ellos.

Fuera de la división, el segundo párrafo se muestra de color rojo, mientras que el primero no porque los párrafos de dentro de la división no son sus hermanos.

```
div {  
  border: red 3px solid;  
  margin: 10px;  
  padding: 10px;  
}  
  
p ~ p {  
  color: red;  
}
```

Esto es un párrafo <p>.

- Esto es una lista

Esto es un párrafo <p> dentro de una lista

Esto es otro párrafo <p>.

Esto no es un párrafo <p>, sino una dirección <address>.

Esto es otro párrafo <p>.

Esto es otro párrafo <p>.

7.3.4 SELECTOR ATRIBUTO

Hay distintas formas de seleccionar elementos con determinados atributos:

- `selector[atributo]`
- `selector[atributo="valor"]`
- `selector[atributo~="valor"]`
- `selector[atributo*="valor"]`
- `selector[atributo | ="valor"]`

SELECTOR[ATRIBUTO]

Si se escribe una etiqueta seguida del nombre de un atributo entre corchetes, las propiedades afectan a todos los elementos que tengan establecidos ese atributo.

Ej.: los párrafos con clase, independientemente de la clase asignada, se muestran en mayúsculas.

ESTO ES UN PÁRRAFO <P> DE CLASE P1.

ESTO ES UN PÁRRAFO <P> DE CLASE P2.

Esto es un párrafo <p> sin clase.

```
p.p1 {  
  color: red;  
}  
  
p.p2 {  
  color: blue;  
}  
  
p[class] {  
  text-transform: uppercase;  
}
```

SELECTOR[ATRIBUTO="VALOR"]

Si se escribe una etiqueta seguida del nombre de un atributo igual a un valor entre corchetes, las propiedades afectan a todos los elementos que tengan establecido ese atributo exactamente con ese valor.

Ej.: el párrafo con dos clases (asignadas en el mismo orden en el que aparecen entre corchetes en la hoja de estilo), se muestran en mayúsculas.

Y los párrafos con dos clases se ven de color azul porque las clases no se aplican en el orden en el que aparecen en el atributo clase, sino en el orden en que aparecen en la hoja de estilo.

Esto es un párrafo <p> de clase p1.

Esto es un párrafo <p> de clase p2.

ESTO ES UN PÁRRAFO <P> CON CLASE P1 Y P2 (EN ESE ORDEN).

Esto es un párrafo <p> con clase p2 y p1 (en ese orden).

```
p.p1 {  
  color: red;  
}  
  
p.p2 {  
  color: blue;  
}  
  
p[class="p1 p2"] {  
  text-transform: uppercase;  
}
```

SELECTOR[ATRIBUTO~="VALOR"]

Si se escribe una etiqueta **selector[atributo~="valor"]**, las propiedades afectan a todos los elementos que tengan en ese atributo una lista de palabras separadas por espacios, una de las cuales sea ese valor.

SELECTOR[ATRIBUTO*="VALOR"]

Si se escribe una etiqueta **selector[atributo*="valor"]**, las propiedades afectan a todos los elementos que contengan ese valor en el valor de ese atributo.

SELECTOR[ATRIBUTO | ="VALOR"]

Si se escribe una etiqueta **selector[atributo | ="valor"]**, las propiedades afectan a todos los elementos que tengan establecido ese atributo con un valor que comience por ese valor seguido de un guion -.

Este selector está pensado para seleccionar elementos con diferentes variantes de un mismo idioma.

7.4 PSEUDOSELECTORES

Existen 2 tipos de pseudoselectores:

- **Pseudoclases:** que sirven para modificar el estilo de un elemento que puede presentar varios estados.
- **Pseudoelementos:** que sirven para modificar aquellos elementos especiales que no pueden ser modificados a través de otros selectores ni de pseudoclases.

7.4.1 PSEUDOCCLASES

Existen ciertos tipos de elementos HTML que pueden presentar más de un estado.

Por ejemplo, el elemento `<a>` que puede tener estos estados:

<i>a:link</i>	Enlaces que no han sido visitados por el usuario.
<i>a:visited</i>	Enlaces visitados al menos una vez por el usuario.
<i>a:hover</i>	El elemento se activa, modifica su estilo, cuando cualquier elemento apuntador pasa por encima de dicho elemento.
<i>a:active</i>	Este estado se activa cuando el usuario activa un elemento, normalmente, al pulsar con el ratón sobre el elemento.
<i>a:focus</i>	El elemento se activa cuando tiene el foco del navegador sobre él, por ejemplo, al seleccionar un elemento en un formulario.

Para asignar un estilo concreto al cambiar de un estado a otro se usará la siguiente sintaxis:

```
a:link {  
    font-family: sans-serif;  
}  
a:hover {  
    color:red;  
}  
a:visited {  
    color:gray;  
}
```

Otra de las pseudoclasas más comunes es la relativa a la posición de un elemento en una lista o grupo de elementos hermanos.

En este caso, en función del lugar del elemento tomará un valor o estilo u otro.

Estados posibles para elementos de un mismo tipo en un grupo de "hermanos"

<i>:first-child</i>	Selecciona el primer elemento de un grupo de elementos "hermanos".
<i>:first-of-type</i>	Se activa si es el primer elemento de la lista.
<i>:last-of-type</i>	Se activa si es el último elemento de la lista.
<i>:only-child</i>	Se activa si es el único elemento de la lista.
<i>:empty</i>	Se activa si la lista está vacía.

```
<ul>
  <li>Nombre y cantidad</li>
  <li>Pepe. 5</li>
  <li>María. 8</li>
  <li>Manuel. 3</li>
  <li>Total. 16</li>
</ul>
```

```
li:first-child {
  font-size:22px; color:#333399; list-style-type: none;
}
li:last-of-type {
  font-size:16px; color:#333399 ; font-weight:bolder;
}
```

ACTIVIDAD 9 Y 9-2

7.4.2 PSEUDOELEMENTOS

Permiten modificar el estilo de determinados elementos especiales, que no pueden ser modificados a través de selectores ni de pseudoclases.

Normalmente, no describen un estado, sino que añaden estilos a una parte concreta del documento.

Los pseudoelementos más comunes son first-line, first-letter, :before y :after.

Sintaxis:

```
selector::pseudo-elemento {  
  propiedad:valor;  
}
```


:FIRST-LINE

Selecciona la primera línea de texto de un elemento.

Ej.: la primera línea será de color rojo.

```
p::first-line {  
    color:red;  
}
```

Este pseudoselector solo se emplea en elementos de bloque y tablas.

:FIRST-LETTER

Selecciona la primera letra de la primera línea de texto de un elemento.

Ej.: la primera letra será de color verde.

```
p::first-letter {  
    color:green;  
}
```



:BEFORE Y :AFTER

Se utilizan para añadir contenido al documento original.

En el caso de :before, se añade al principio, y en el caso de :after, después.



En la sintaxis de uso debe aparecer la propiedad **content**, donde se indica el nuevo contenido que se va a incorporar.

```
h1::before {  
    content: "INICIO.-";  
}
```

```
h1::after {  
    content: ".-FIN";  
}
```

[Amplia tu información sobre el pseudoelemento : after](#)

ACTIVIDAD 10

8. TIPOS DE ELEMENTOS CSS. DE BLOQUE Y EN LÍNEA

Un elemento de bloque de CSS siempre comienza en una línea nueva generando saltos de línea, antes y después de él, mientras que los elementos en línea de CSS no lo hacen, aparecen en el flujo de la línea y sólo pasarán a otra línea si no tienen espacio.

En (X)HTML, los párrafos (**p**), cabeceras (como **h1**), listas (**ol**, **ul**, **dl**) y contenedores (**div**) son los elementos de bloque más comunes.



Mientras que, el texto enfatizado (**em**), las anclas y enlaces (**a**) y los elementos (**span**) son los elementos en línea más comunes.

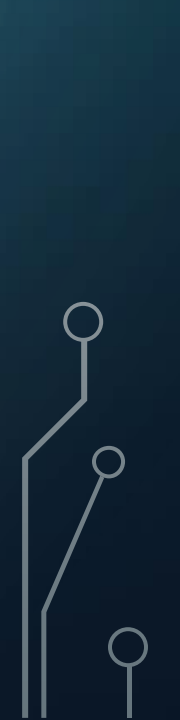


Con CSS podrás indicarle al navegador cómo quieres que se vea en el documento empleando para ello los atributos **block** e **inline** de la propiedad **display**, independientemente de que un elemento sea de bloque o en línea.



La propiedad `display` tiene un valor por defecto y los elementos pueden ser de línea o de bloque.

ELEMENTOS EN LÍNEA

- Ocupan el espacio necesario para mostrar su contenido.
 - No tienen dimensiones modificables (alto y ancho).
 - Permiten otros elementos a su lado.
 - `Padding` y `margin` solo empujan a elementos adyacentes en horizontal, nunca en vertical.
- 
- 

ELEMENTOS EN BLOQUE

- Ocupan todo el ancho disponible.
- Tienen dimensiones modificables (alto y ancho).
- No permiten otros elementos a su lado (aunque especifique un ancho, sigue ocupando todo el espacio disponible).

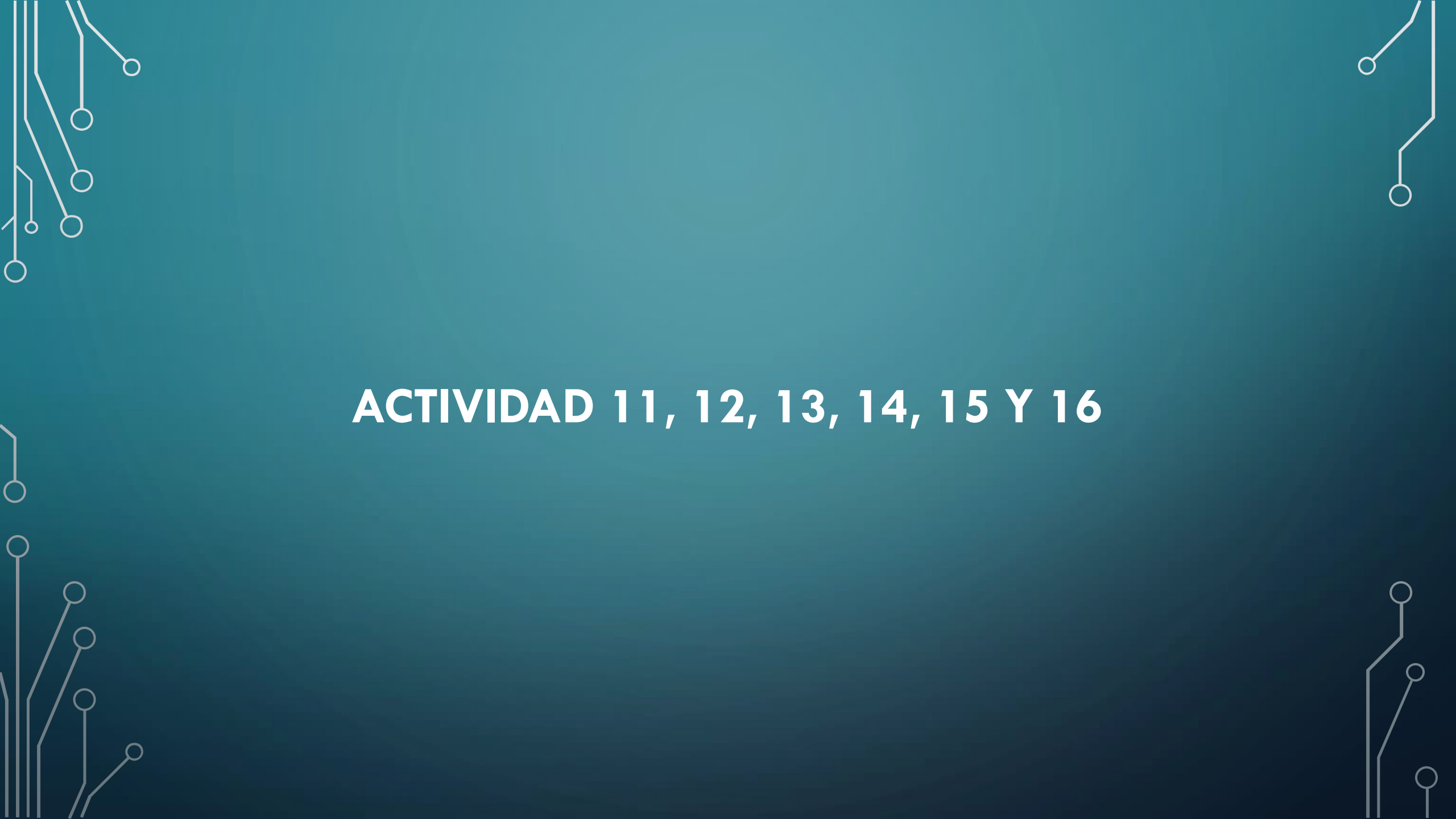
ELEMENTOS INLINE-BLOCK

- Mezcla de en línea y bloque.
- Tienen dimensiones modificables (alto, ancho).
- Permiten otros elementos a su lado.
- Si no especifican dimensiones, ocupan lo que ocupe su contenido.

Consideraciones sobre los elementos inline-block

- vertical-align alinea elementos entre ellos (no su contenido).

Espacios no deseables entre elementos y en imágenes dentro de contenedores

The background is a dark teal gradient. In the corners, there are white line-art illustrations of circuit boards or neural networks, with lines and small circles representing nodes and connections.

ACTIVIDAD 11, 12, 13, 14, 15 Y 16

9. BOX-SIZING

La propiedad box-sizing **calcula el alto y ancho** total de un elemento.

Por defecto, en el modelo de caja de CSS, el ancho y alto asignado a un elemento es aplicado solo al contenido de la caja del elemento.

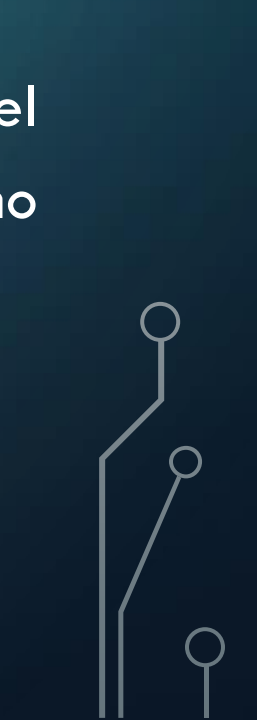
Si el elemento tiene algún borde (border) o relleno (padding), este es entonces añadido al ancho y alto hasta alcanzar el tamaño de la caja que es desplegada en pantalla $\Rightarrow$ cuando se definen el ancho y alto, se tiene que ajustar el valor definido para permitir cualquier borde o relleno que se pueda añadir.



Se puede usar la propiedad `box-sizing` para ajustar ese comportamiento:

- **content-box:** es el comportamiento por defecto para el tamaño de la caja (`box-sizing`).

Si se define el ancho de un elemento en 100 píxeles, la caja del contenido del elemento tendrá 100 píxeles de ancho, y el ancho de cualquier borde o relleno se añadirá al ancho final desplegado.



- **border-box:** le dice al navegador que tome en cuenta cualquier valor que se especifique de borde o de relleno para el ancho o alto de un elemento.

Si se define un elemento con un ancho de 100 pixeles, esos 100 pixeles incluirán cualquier borde o relleno que se añada, y la caja de contenido se encogerá para absorber ese ancho extra, lo que suele facilitar dimensionar elementos.

Comprueba el funcionamiento de box-sizing:

```
<div class="content-box">Contenido de la caja</div>  
<br>  
<div class="border-box">Borde de la caja</div>
```

```
div {  
  width: 160px;  
  height: 80px;  
  padding: 20px;  
  border: 8px solid red;  
  background: yellow;  
}  
  
.content-box {  
  box-sizing: content-box;  
  /* Ancho total: 160px + (2 * 20px) + (2 * 8px) = 216px  
  Altura total: 80px + (2 * 20px) + (2 * 8px) = 136px  
  Ancho de la caja de contenido: 160px  
  Altura de la caja de contenido: 80px */  
}  
  
.border-box {  
  box-sizing: border-box;  
  /* Ancho total: 160px  
  Altura total: 80px  
  Ancho de la caja de contenido: 160px - (2 * 20px) - (2 * 8px) = 104px  
  Altura de la caja de contenido: 80px - (2 * 20px) - (2 * 8px) = 24px */  
}
```

ACTIVIDAD 17, 18 Y 19

10. VALIDACIONES DE CÓDIGO BIEN FORMADO

La validación es muy importante, ya que asegurarnos de que el código HTML es válido puede ahorrar muchas horas de comprobaciones.

Si el código HTML contiene imperfecciones, hay muchas posibilidades de que el JavaScript o CSS no funcionen de la manera esperada, ya que ambos dependen de los elementos HTML y sus atributos.



Cuanto más te ajustes a las especificaciones del estándar, mejor resultado obtendrás entre los diferentes tipos de navegadores.



El consorcio **W3C**, el cual diseñó el lenguaje HTML, **desarrolló un validador que permite chequear si una página web cumple las especificaciones indicadas por el elemento DOCTYPE** que se incluye al principio de cada página web que se realiza.



El validador ayuda a realizar páginas más consistentes.

[Servicio de validación CSS](#)

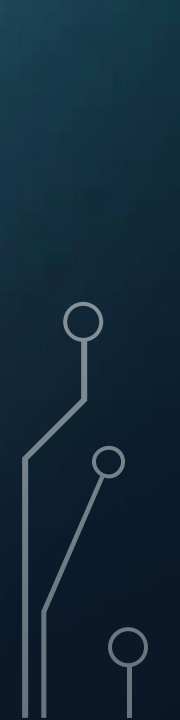
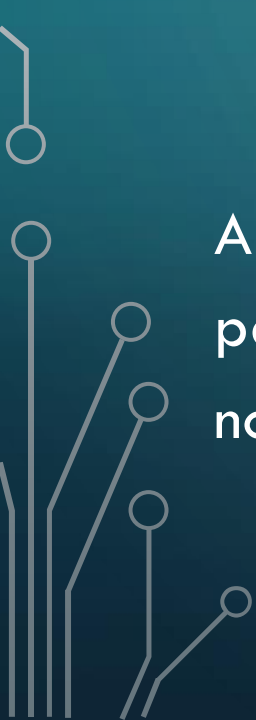
[Servicio de validación de marcas](#)





Ofrece tres formas de introducción del código para la validación:

- Copiando y pegando el código en un formulario
- Enviando el fichero .html
- Indicando la dirección URL dónde se está la página web



Al pulsar el botón de chequear el validador devolverá los errores encontrados, para que se realicen los cambios necesarios y volver a validar de nuevo hasta que no haya errores.



Más información sobre las diferentes especificaciones de **W3C**:

- <http://www.w3.org/TR/html5/>
- <http://www.w3.org/TR/html4/>
- <http://www.w3.org/TR/xhtml1/>

Especificación sobre CSS:

- <http://www.w3.org/TR/CSS21/>
- 
- 

ACTIVIDAD 20